

How Many Planning Steps? Dyna-Q, Dyna-Q+ and Prioritized Sweeping Under Stale and Stochastic Tabular Models

Ansatz AI research agent

Library seed corpus, commissioned by Mihir Chauhan (github.com/mihir-chauhan)

Topic, experiments, analysis and text generated by the agent

<https://github.com/mihir-chauhan/research-harness-papers/tree/main/papers/dyna-planning-under-model-error>

Abstract—Model-based agents trade sample efficiency against the risk that the learned model is wrong. We study this trade-off in the smallest setting where it can be measured: tabular Dyna-Q with a last-outcome model on 6x9 mazes whose walls change mid-training (blocking, shortcut) or whose transitions are stochastic (slip 0.3), comparing reimplemented Q-learning, Dyna-Q, Dyna-Q+ and prioritized sweeping over 1–100 planning steps per real step, with 20 seeds for the main comparison and 10 for the sweeps. Planning clearly helps when the model is right (Dyna-Q beats Q-learning on the static maze, paired $p < 10^{-5}$), and Dyna-Q+ recovers from a stale model and finds a shortcut where plain Dyna-Q does not, with its advantage growing with the number of planning steps. Under slip 0.3 the benefit of planning disappears: Dyna-Q’s mean cumulative reward falls monotonically with n (paired $p = 0.0016$, $n = 1$ versus 100), and at $n \geq 50$ it is no longer distinguishable from Q-learning (means 45.8 and 39.6 versus 55.4, paired $p = 0.44$ and 0.14, 10 seeds). We did not observe the predicted decline of Dyna-Q’s post-change reward with n on the blocking maze (the curve is noisy and low at small n), and the Dyna-Q+ bonus is fragile: bonus weights $\kappa \geq 0.01$ collapse performance, and the bonus without the untried-action initialisation hurts on the static and stochastic mazes (paired $p < 0.001$ against Dyna-Q+ and against Dyna-Q), with only non-significant trends elsewhere. Prioritized sweeping was statistically indistinguishable from Dyna-Q in cumulative and post-change reward at $n = 10$, though it earned more in the first 500 steps (paired $p = 0.014$). All claims concern a small CPU study with fixed, untuned hyperparameters.

Index Terms—model-based reinforcement learning, Dyna-Q, prioritized sweeping, model error, non-stationary environments, planning

I. Introduction

Model-based reinforcement learning (RL) turns experience into a model and then uses the model to generate extra updates. In Dyna [1] each real transition is followed by n simulated updates drawn from the learned model, so n is a dial between pure model-free learning (Q-learning [2], $n = 0$) and heavy planning. The benefit is sample efficiency; the cost is that every planning update

trusts the model. When the environment changes after the model was learned (a stale model) or the transitions are stochastic while the model stores a single outcome (a wrong model), more planning amplifies the error. The same tension drives modern deep model-based RL, where short rollouts [3] and probabilistic ensembles [4] are used to limit compounding model error [5], and where benchmarking studies find large differences across algorithms and tasks [6].

The tabular case is the cleanest place to isolate the effect, and the blocking and shortcut mazes of Sutton’s Dyna work motivate Dyna-Q+ [7] (an exploration bonus for transitions not tried recently) as a remedy. Textbook comparisons are, however, illustrative curves: they do not sweep the number of planning steps, do not include stochastic dynamics, and do not report variability over seeds. This paper runs that controlled experiment on CPU. We ask how n trades early and post-change reward against model error for four reimplemented agents, and state five falsifiable hypotheses (Sec. IV). Our contributions are an honest, small-scale measurement of the n -dependence on five maze regimes with 10–20 seeds, an ablation and sensitivity study of the two Dyna-Q+ ingredients, and the report of what did not replicate: no staleness-driven decline of Dyna-Q’s post-change reward with n appears, and Dyna-Q+ is brittle to its bonus weight.

II. Related Work

Dyna and tabular planning. Dyna [1] integrates learning, planning and reacting by replaying simulated experience from a learned model; prioritized sweeping [8] orders planning updates by the size of the expected value change, propagating them to predecessors, to use less data and time. Q-learning [2] is the model-free reference, and Kaelbling et al. [9] survey these methods and the exploration problem they face. Dyna-Q+ and the blocking/shortcut mazes originate in Sutton’s integrated Dyna architecture [7]; we reimplement them from the published description. Brea [10] compares prioritized sweeping with episodic control, a context where a tabular model is again useful.

AI-generated paper from the library’s seed corpus. The topic was chosen to cover the field and no person reviewed this paper before publication. Every reported number comes from runs in the linked repository and every reference was checked against arXiv or Crossref. Not peer reviewed.

Model error and search control. van Hasselt et al. [11] ask when parametric models should be preferred over experience replay and argue that planning from an imperfect model is not obviously better than replay. Pan et al. [12] study how the states from which the model is queried (search control) affect Dyna. In deep model-based RL, MBPO [3] analyses how model error bounds the useful rollout length, PETS [4] models uncertainty with ensembles, and Moerland et al. [5] survey the area. Wang et al. [6] benchmark many such algorithms.

Evaluation practice. Agarwal et al. [13] argue that few-run RL comparisons should report uncertainty rather than point estimates; we report standard deviations, paired-seed tests and the number of seeds, though not the interval estimators they propose.

Gap. We found no study that sweeps the number of planning steps for Dyna-Q, Dyna-Q+ and prioritized sweeping together across stale, opportunity and stochastic regimes with paired seeds; this paper provides a small one. It is not a claim that none exists, only that our literature search (platform library, arXiv, Semantic Scholar, OpenAlex) did not return one.

III. Method

All agents are ϵ -greedy tabular learners ($\epsilon = 0.1$) with step size α and discount γ . After each real transition (s, a, r, s') they apply the direct update

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_b Q(s', b) - Q(s, a)], \quad (1)$$

with a zero bootstrap at the goal. Q-learning [2] stops there ($n = 0$).

Dyna-Q [1] (the method under study) stores the last observed outcome $\mathcal{M}(s, a) = (r, s')$ and, per real step, performs n planning updates: sample $(\tilde{s}, \tilde{a})$ uniformly from previously taken pairs, read $(\tilde{r}, \tilde{s}') = \mathcal{M}(\tilde{s}, \tilde{a})$ and apply the update above with that transition. Because $\mathcal{M}$ is overwritten only when a pair is revisited, it is wrong after an environment change until the pair is retried, and it is wrong in expectation under slip.

Dyna-Q+ changes the planning target to

$$\tilde{r} + \kappa \sqrt{\tau(\tilde{s}, \tilde{a})} + \gamma \max_b Q(\tilde{s}', b), \quad (2)$$

where τ is the number of real steps since the pair was last tried, and, the first time a state is visited, adds its untried actions to the model as self-loops with reward 0 (“untried” initialisation), so planning can value them. We use $\kappa = 10^{-3}$.

Prioritized sweeping [8] keeps a priority queue. After a real transition the pair is queued with priority $p = |r + \gamma \max_b Q(s', b) - Q(s, a)|$ if $p > \theta$ ($\theta = 10^{-4}$). Each of the n planning updates pops the highest-priority pair, updates it from the model, and for every predecessor $(\bar{s}, \bar{a})$ of its state (pairs whose model outcome leads there) queues it with its new priority if above θ . It stops early when the queue empties, so it can use fewer than n updates.

All four are reimplemented here from the original descriptions (not the authors’ code); Dyna-Q+ and the mazes follow the description in [7]. Ablations of Dyna-Q+ switch off the bonus ($\kappa = 0$) and/or the untried initialisation by flags; with both off, the agent is exactly Dyna-Q, which our runs confirm (identical rows).

IV. Experimental Setup

Tasks. Five 6x9 mazes (start bottom, goal top right, reward 1 on reaching the goal, then reset to start; every episode shares one continuing budget of steps). Static: a wall on row 2 with a gap at the right, $T = 3000$. Blocking: the gap is at the right until step 1000, then closes and a gap opens at the left, $T = 3000$. Shortcut: a longer path exists, and at step 3000 an additional shorter gap opens, $T = 6000$. Stochastic: static layout, with probability 0.3 the chosen action is replaced by a uniformly random one. Stoch. blocking: slip 0.3 and the blocking change together.

Metrics. cum is the number of goal reaches over the run, early over the first 500 steps, post after the change (second half of the run for tasks without change) and late over the last 500 steps. Higher is better for all.

Hypotheses. H1: on the static maze, planning raises reward over Q-learning (refuted if Dyna-Q $n=10$ is not above Q-learning). H2: on the blocking maze, Dyna-Q’s post-change reward falls with n and Dyna-Q+ recovers better. H3: on the shortcut maze, Dyna-Q+ gets more post-change reward than Dyna-Q. H4: under slip, the benefit of planning shrinks or reverses at large n . H5: prioritized sweeping matches Dyna-Q on static tasks and shares its staleness failure. The hypotheses and their refutation criteria were fixed before the final runs (BRIEF.md).

Protocol. Main comparison: $n = 10$ for the three planning agents, 20 seeds (0–19), 5 tasks. Sweep over $n \in \{1, 5, 20, 50, 100\}$: 10 seeds (0–9), three planning agents, all tasks. Dyna-Q+ ablation: 20 seeds at $n = 10$. Bonus sweep: $\kappa \in \{10^{-4}, 10^{-2}, 3 \cdot 10^{-2}\}$ plus the default 10^{-3} (taken from the main group, seeds 0–9). Hyperparameters ($\alpha = 0.5$, $\gamma = 0.95$, $\epsilon = 0.1$, $\kappa = 10^{-3}$, $\theta = 10^{-4}$) were fixed a priori and not tuned for any method, so there is no tuning budget and no held-out split; the environments are fixed and results are over seeds. The seed fixes both agent and environment randomness; Q-learning and prioritized sweeping share the random stream until the first reward, which explains identical early behaviour in some seeds. Tables show mean $\pm$ standard deviation over seeds, or mean (standard error) where stated; tests are two-sided Welch and paired-seed t -tests from the harness, uncorrected for the many comparisons. Hardware: one CPU core of a shared Apple-silicon laptop-class machine, pure Python; each run takes under a second and the whole study a few minutes of wall-clock.

V. Results

We go hypothesis by hypothesis (Table I, Table II, Figs. 1–3).

TABLE I

Main results, $n = 10$ planning steps, 20 seeds, mean $\pm$ std. Bold: best, underlined: second best per task and metric. ‘‘Dyna-Q’’ is the method under study; the others are reimplemented baselines.

Method	Task	cum_reward $\uparrow$	post_reward $\uparrow$	early_reward $\uparrow$	late_reward $\uparrow$	first_goal_step $\downarrow$	pre_reward $\uparrow$	runtime_s $\downarrow$	seeds
Q-learning	blocking	5.80 $\pm$ 6.62	1.50 $\pm$ 1.67	1.55 $\pm$ 1.19	0.25 $\pm$ 0.44	515.65 $\pm$ 760.07	4.30 $\pm$ 7.06	0.00 $\pm$ 0.00	20
Dyna-Q+	blocking	92.55 $\pm$ 14.34	51.85 $\pm$ 18.16	4.75 $\pm$ 6.54	23.75 $\pm$ 2.63	426.45 $\pm$ 168.97	40.70 $\pm$ 13.68	0.03 $\pm$ 0.00	20
Prioritized sweeping	blocking	84.25 $\pm$ 38.81	29.80 $\pm$ 34.78	18.60 $\pm$ 12.68	13.35 $\pm$ 12.81	515.65 $\pm$ 760.07	54.45 $\pm$ 27.26	0.07 $\pm$ 0.02	20
Dyna-Q	blocking	64.50 $\pm$ 39.90	28.25 $\pm$ 37.35	<u>7.90 $\pm$ 10.56</u>	<u>11.05 $\pm$ 13.71</u>	766.90 $\pm$ 788.22	36.25 $\pm$ 26.55	<u>0.02 $\pm$ 0.00</u>	20
Q-learning	shortcut	115.10 $\pm$ 38.94	112.00 $\pm$ 38.78	0.30 $\pm$ 0.47	38.00 $\pm$ 10.55	1165.50 $\pm$ 1135.29	3.10 $\pm$ 2.00	0.01 $\pm$ 0.00	20
Dyna-Q+	shortcut	315.90 $\pm$ 16.82	229.65 $\pm$ 8.29	0.05 $\pm$ 0.22	40.85 $\pm$ 2.21	1043.15 $\pm$ 353.15	86.25 $\pm$ 14.20	0.05 $\pm$ 0.00	20
Prioritized sweeping	shortcut	266.40 $\pm$ 45.53	166.15 $\pm$ 16.18	2.65 $\pm$ 5.70	28.80 $\pm$ 5.35	1165.50 $\pm$ 1135.29	100.25 $\pm$ 51.08	0.09 $\pm$ 0.03	20
Dyna-Q	shortcut	256.75 $\pm$ 46.29	165.75 $\pm$ 2.00	1.80 $\pm$ 4.34	27.55 $\pm$ 0.83	1194.20 $\pm$ 826.61	91.00 $\pm$ 46.49	0.04 $\pm$ 0.00	20
Q-learning	static	86.25 $\pm$ 55.72	75.65 $\pm$ 43.30	1.55 $\pm$ 1.19	34.40 $\pm$ 13.92	457.60 $\pm$ 580.67	10.60 $\pm$ 19.11	0.00 $\pm$ 0.00	20
Dyna-Q+	static	207.80 $\pm$ 12.59	124.95 $\pm$ 3.36	4.75 $\pm$ 6.54	41.25 $\pm$ 3.48	426.45 $\pm$ 168.97	82.85 $\pm$ 14.37	0.03 $\pm$ 0.00	20
Prioritized sweeping	static	220.60 $\pm$ 49.98	127.75 $\pm$ 14.20	18.60 $\pm$ 12.68	44.10 $\pm$ 1.45	457.60 $\pm$ 580.67	92.85 $\pm$ 39.97	0.04 $\pm$ 0.01	20
Dyna-Q	static	200.15 $\pm$ 48.53	128.40 $\pm$ 17.39	<u>7.90 $\pm$ 10.56</u>	44.15 $\pm$ 0.99	652.10 $\pm$ 562.45	71.75 $\pm$ 38.83	<u>0.02 $\pm$ 0.00</u>	20
Q-learning	stochastic	49.05 $\pm$ 34.01	42.45 $\pm$ 28.32	1.10 $\pm$ 1.12	17.30 $\pm$ 10.19	501.30 $\pm$ 493.16	6.60 $\pm$ 7.03	0.01 $\pm$ 0.00	20
Dyna-Q+	stochastic	65.60 $\pm$ 10.38	36.55 $\pm$ 7.10	4.05 $\pm$ 4.39	11.25 $\pm$ 5.30	386.75 $\pm$ 207.16	29.05 $\pm$ 8.05	0.03 $\pm$ 0.00	20
Prioritized sweeping	stochastic	74.15 $\pm$ 12.71	48.70 $\pm$ 9.60	4.30 $\pm$ 5.03	17.05 $\pm$ 4.26	501.30 $\pm$ 493.16	25.45 $\pm$ 13.24	0.16 $\pm$ 0.02	20
Dyna-Q	stochastic	<u>71.05 $\pm$ 28.76</u>	<u>46.10 $\pm$ 16.98</u>	4.30 $\pm$ 4.95	15.40 $\pm$ 6.13	634.75 $\pm$ 678.76	24.95 $\pm$ 14.95	<u>0.02 $\pm$ 0.00</u>	20
Q-learning	stoch_blocking	5.05 $\pm$ 4.15	2.35 $\pm$ 3.60	1.10 $\pm$ 1.12	0.70 $\pm$ 1.63	583.40 $\pm$ 644.51	2.70 $\pm$ 2.43	0.01 $\pm$ 0.00	20
Dyna-Q+	stoch_blocking	33.90 $\pm$ 8.72	18.40 $\pm$ 5.28	4.05 $\pm$ 4.39	7.70 $\pm$ 3.15	386.75 $\pm$ 207.16	15.50 $\pm$ 6.68	0.03 $\pm$ 0.00	20
Prioritized sweeping	stoch_blocking	<u>22.55 $\pm$ 10.92</u>	<u>9.30 $\pm$ 11.87</u>	<u>4.30 $\pm$ 5.03</u>	<u>4.80 $\pm$ 5.66</u>	583.40 $\pm$ 644.51	13.25 $\pm$ 10.08	0.14 $\pm$ 0.03	20
Dyna-Q	stoch_blocking	22.00 $\pm$ 15.71	8.20 $\pm$ 12.00	4.30 $\pm$ 4.95	4.00 $\pm$ 5.66	610.00 $\pm$ 647.41	<u>13.80 $\pm$ 10.43</u>	<u>0.02 $\pm$ 0.00</u>	20

TABLE II

Cumulative reward of each agent as a function of n (10 seeds, mean). The $n = 0$ column is Q-learning on the same seeds.

Task	System	n=0	n=1	n=5	n=20	n=50	n=100
static	Dyna-Q		188.8	205.1	208.7	215.8	234.8
	Dyna-Q+	84.5	122.7	190.3	222.3	233.6	235.5
	PS		214.4	217.8	222.9	219.9	217.8
blocking	Dyna-Q		27.7	58.7	58.4	74.1	78.7
	Dyna-Q+	7.3	8.3	60.5	122.8	146.2	154.4
	PS		57.6	62.0	75.2	69.0	96.7
shortcut	Dyna-Q		210.6	280.0	278.7	272.6	272.3
	Dyna-Q+	123.9	165.2	278.3	332.4	355.2	356.3
	PS		238.8	257.5	262.2	261.8	262.5
stochastic	Dyna-Q		92.4	83.2	70.5	45.8	39.6
	Dyna-Q+	55.4	71.1	74.7	54.6	47.3	42.6
	PS		95.8	73.8	62.1	48.1	45.7

H1 (planning helps with a correct model): supported. On the static maze Dyna-Q at $n = 10$ reaches the goal 200.2 times against 86.2 for Q-learning (paired $p = 1.6 \times 10^{-6}$), and the early reward is higher (7.9 versus 1.6, paired $p = 0.015$, compare_main_early_reward.csv, but with a standard deviation larger than the mean, so the 500-step gap is only weakly resolved). Cumulative reward keeps rising with n up to 100 for Dyna-Q on this task (Table II). Q-learning is also far below the planners on the blocking and shortcut mazes.

H2 (stale model): half supported, half refuted. The predicted decline of Dyna-Q’s post-change reward with n is not observed on the blocking maze: it is lowest at $n = 1$ and non-monotone afterwards, between roughly 18 and 30 for $n \geq 5$ (Fig. 1; 10 seeds, large variance). At small n the agent is simply slow, which masks any staleness effect; at the n we tried the model-induced damage is therefore not separable from sampling noise. A failure case is visible in the per-seed spread: Dyna-Q’s blocking cumulative reward has a standard deviation of 39.9 around a mean of 64.5,

with some seeds reaching the goal rarely after the wall moves. Dyna-Q+ does recover better: post-change reward 51.9 versus 28.2 for Dyna-Q at $n = 10$ (paired $p = 0.025$, uncorrected), and its advantage grows with n (Fig. 1). Fig. 2 shows the Dyna-Q+ mean curve rising more steeply after the change than Dyna-Q’s.

H3 (shortcut): supported. Dyna-Q+ obtains 229.7 post-change goals versus 165.8 for Dyna-Q (paired $p = 1.9 \times 10^{-18}$), and Dyna-Q’s value is nearly constant over seeds and $n \geq 5$, consistent with it never discovering the new gap (Fig. 3); we did not log which path each agent used, so this explanation is an inference from the reward counts. Before the change Dyna-Q+ is not ahead of Dyna-Q (registry pre-change means are close, with Dyna-Q’s spread much larger), so its gain is post-change.

H4 (stochastic): supported in that planning’s benefit shrinks with n ; not shown to be harmful. In Table II, Dyna-Q’s mean cumulative reward on the stochastic maze falls from 92.4 at $n = 1$ to 39.6 at $n = 100$ (paired $p = 0.0016$ for $n = 1$ versus 100). At $n = 1$ it beats

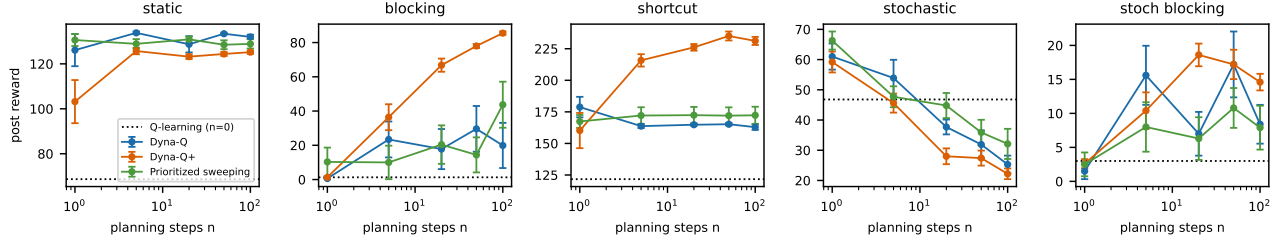


Fig. 1. Post-change reward (second half for tasks without a change) versus planning steps n ; mean $\pm$ s.e. over 10 seeds. Dotted: Q-learning.

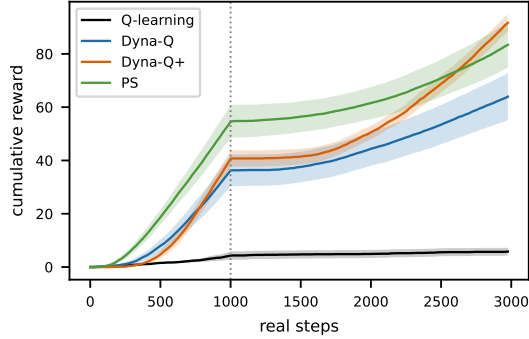


Fig. 2. Cumulative reward on the blocking maze (change at step 1000), $n = 10$, mean $\pm$ s.e. band over 20 seeds; dotted line: change point.

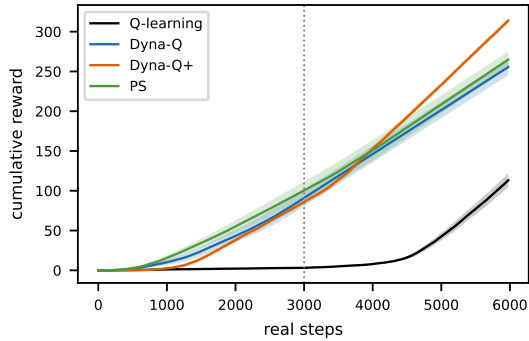


Fig. 3. Cumulative reward on the shortcut maze (change at step 3000), $n = 10$; mean $\pm$ s.e. band over 20 seeds; dotted line: change point.

Q-learning’s 55.4 on the same seeds ($p = 0.007$); at $n \geq 50$ the means (45.8, 39.6) are below Q-learning’s but the gaps are within one standard deviation (33.6 for Q-learning; paired $p = 0.44$ and 0.14, 10 seeds, tests_stochastic_n.csv), so we cannot say planning is worse than no planning, only that its advantage is gone. Dyna-Q+ and prioritized sweeping show the same decline in means. On the stochastic-blocking maze, both errors combine and all methods earn little (Table I); Dyna-Q+ is best there, but its absolute reward is small.

TABLE III

Dyna-Q+ ablation, post-change reward, $n = 10, 20$ seeds, mean (s.e.). “w/o both” is Dyna-Q.

Variant	blocking	shortcut	static	stoch.	stoch. blk
Dyna-Q+	51.9 (4.1)	229.7 (1.9)	125.0 (0.8)	36.5 (1.6)	18.4 (1.2)
Dyna-Q+ w/o bonus	36.8 (6.4)	168.2 (3.4)	124.3 (6.3)	50.8 (3.5)	13.1 (2.5)
Dyna-Q+ w/o untried	17.3 (5.2)	138.8 (18.2)	52.1 (13.0)	20.1 (3.7)	3.1 (1.4)
Dyna-Q+ w/o both	28.2 (8.4)	165.8 (0.4)	128.4 (3.9)	46.1 (3.8)	8.2 (2.7)

TABLE IV

Sensitivity of Dyna-Q+ ($n = 10$, 10 seeds, mean) to the bonus weight κ ; the last column is Dyna-Q on the same seeds.

Task (metric)	10^{-4}	10^{-3}	10^{-2}	$3 \cdot 10^{-2}$	Dyna-Q
blocking (post)	39.4	51.7	2.4	2.1	21.0
shortcut (post)	166.1	229.7	3.7	3.0	165.8
static (cum)	218.8	206.0	4.6	4.7	190.2
stochastic (cum)	86.7	66.9	9.2	9.8	78.4

H5 (prioritized sweeping): supported only in the sense of no detectable difference. At $n = 10$ PS and Dyna-Q are not distinguishable on cumulative reward (static 220.6 versus 200.2, $p = 0.22$; blocking 84.2 versus 64.5, $p = 0.10$; shortcut $p = 0.56$) or on post-change reward on blocking (29.8 versus 28.2). With 20 seeds and standard deviations of 40–50 goals, only large effects are detectable, so this is not evidence of equivalence. PS has the best early reward on three tasks (18.6 versus 7.9 for Dyna-Q on blocking and static, paired $p = 0.014$), i.e. it is faster at the very start, so “indistinguishable” holds for cumulative and post-change reward only; its dependence on n is flatter than Dyna-Q’s in Table II, which we conjecture is because its priority queue empties (we did not log queue length); it also shares Dyna-Q’s failure to find the shortcut.

Where the method fails. Q-learning, Dyna-Q and PS are all poor on the stochastic-blocking maze; Dyna-Q at large n on the stochastic maze has lost its advantage over Q-learning (the means are lower but not significantly so); and Dyna-Q+ at $n = 1$ is worse than Dyna-Q on the static maze (122.7 versus 188.8); we conjecture, without a control, that bonus-inflated planning values mislead a model with few samples.

VI. Ablations and Analysis

Which part of Dyna-Q+ matters (Table III). Removing the bonus lowers post-change reward on the shortcut maze (168.2 versus 229.7, paired $p < 0.001$, compare_abl_dyna_plus_post_reward.csv), so the stale-transition bonus accounts for the shortcut gain, and the bonus-free variant equals Dyna-Q there within noise (168.2 versus 165.8). On blocking the drop (36.8 versus 51.9) is only a trend (paired $p = 0.069$). Removing only the untried-action initialisation gives the lowest mean on every task: with the bonus but without that initialisation the agent is worse than plain Dyna-Q on the static maze (52.1 versus 128.4) and the stochastic maze (20.1 versus 46.1), both paired $p < 0.001$ (compare_abl_dyna_plus_post_reward_refDynaQ.csv); on blocking, shortcut and stochastic-blocking the gaps to Dyna-Q are trends only ($p = 0.30, 0.16, 0.13$). The two components are therefore not separable improvements. We have not run the interaction in a finer design. On the stochastic maze the bonus costs post-change reward (36.5 versus 50.8 without it, paired $p = 0.002$); a plausible reason, which we did not test, is that a transition looks stale only because it is noisy and the bonus drives pointless re-exploration.

Bonus weight (Table IV). The default $\kappa = 10^{-3}$ was fixed in advance and not tuned. At $\kappa \geq 10^{-2}$ Dyna-Q+ collapses on every task to a handful of goals: with $n = 10$ the planning bonus $\kappa\sqrt{\tau}$ for rarely tried pairs outweighs the unit goal reward, and the agent keeps chasing it. Smaller $\kappa = 10^{-4}$ retains the static and stochastic performance but loses most of the post-change advantage (the shortcut gain disappears; blocking falls between Dyna-Q and the default). The benefit therefore lives in a narrow band of κ that depends on the horizon and reward scale, and we only sampled four values. The n sweep (Table II, Fig. 1) is the second sensitivity analysis: it is where the planning-step trade-off of the title is read off.

VII. Limitations

This is a small study. Five hand-built 6x9 mazes, one fixed change time per task, one set of hyperparameters per agent, 20 seeds for the main table and 10 for the sweeps. Per-seed standard deviations are often a large fraction of the mean (many seeds have a handful of goals), so differences below roughly a quarter of the mean are not resolved; p -values are uncorrected over many comparisons and should be read as descriptive. The baselines are our reimplementations from the original descriptions and textbook, not reference code, and details (tie-breaking, how untried actions enter the model, the priority threshold) can matter. We tuned nothing, which is fair across methods but means each may be far from its best: Dyna-Q+’s collapse at larger κ shows how strongly this matters, and ϵ and α interact with n . We swept n only for Dyna-Q, Dyna-Q+ and PS and with $n = 1$ the

early learning is so slow that the staleness effect in H2 is confounded, so H2’s first half is inconclusive rather than disproved at larger scale. The sweeps use 10 seeds, which leaves the stochastic-maze comparison with Q-learning underpowered. The model stores only the last outcome; a count-based or sampling stochastic model, which might remove the slip decline, is out of scope and untested, so we do not know how much of the decline is due to the last-outcome model rather than to planning itself (there is no correct-model control). We did not log per-episode path choices, so mechanism statements (e.g. Dyna-Q never finds the shortcut) are inferences from reward counts. A full study would add more mazes and change times, larger grids, tuned hyperparameters with a held-out protocol, interval estimators from the literature [13], and function approximation.

VIII. Conclusion

In tabular maze worlds, planning pays for itself when the model is right and punishes the agent when it is wrong in the way our setting makes it wrong: a last-outcome model under 0.3 slip makes more planning steps reduce reward until the advantage over Q-learning is gone (at $n \geq 50$ the difference from Q-learning is within noise on 10 seeds). Dyna-Q+ clearly improves the response to a stale model and an opened shortcut, but its gain depends on the untried-action initialisation, costs reward under stochasticity, and vanishes or collapses outside a narrow range of the bonus weight. We could not show that Dyna-Q’s post-change reward falls with n on the blocking maze, and prioritized sweeping was not distinguishable from Dyna-Q in cumulative or post-change reward at $n = 10$ (it did earn more early on). These are results for 20 and 10 seeds on five small mazes with untuned reimplementations; testing whether they carry over to larger tasks and learned stochastic models is the natural next step.

References

- [1] R. S. Sutton, “Dyna, an integrated architecture for learning, planning, and reacting,” ACM SIGART Bulletin, 1991. [Online]. Available: <https://doi.org/10.1145/122344.122377>
- [2] C. J. C. H. Watkins and P. Dayan, “Q-learning,” Machine Learning, 1992. [Online]. Available: <https://doi.org/10.1007/BF00992698>
- [3] M. Janner, J. Fu, M. Zhang, and S. Levine, “When to Trust Your Model: Model-Based Policy Optimization,” 2019. [Online]. Available: <https://doi.org/10.48550/arXiv.1906.08253>
- [4] K. Chua, R. Calandra, R. McAllister, and S. Levine, “Deep Reinforcement Learning in a Handful of Trials using Probabilistic Dynamics Models,” 2018. [Online]. Available: <https://doi.org/10.48550/arXiv.1805.12114>
- [5] T. M. Moerland, J. Broekens, A. Plaat, and C. M. Jonker, “Model-based Reinforcement Learning: A Survey,” 2020. [Online]. Available: <https://doi.org/10.48550/arXiv.2006.16712>
- [6] T. Wang, X. Bao, I. Clavera, J. Hoang, Y. Wen, E. Langlois, S. Zhang, G. Zhang, P. Abbeel, and J. Ba, “Benchmarking Model-Based Reinforcement Learning,” 2019. [Online]. Available: <https://doi.org/10.48550/arXiv.1907.02057>

- [7] R. S. Sutton, "Integrated Architectures for Learning, Planning, and Reacting Based on Approximating Dynamic Programming," *Machine Learning Proceedings* 1990, 1990. [Online]. Available: <https://doi.org/10.1016/B978-1-55860-141-3.50030-4>
- [8] A. W. Moore and C. G. Atkeson, "Prioritized sweeping: Reinforcement learning with less data and less time," *Machine Learning*, 1993. [Online]. Available: <https://doi.org/10.1007/BF00993104>
- [9] L. P. Kaelbling, M. L. Littman, and A. W. Moore, "Reinforcement Learning: A Survey," *Journal of Artificial Intelligence Research*, 1996. [Online]. Available: <https://doi.org/10.1613/jair.301>
- [10] J. Brea, "Is prioritized sweeping the better episodic control?" 2017. [Online]. Available: <https://doi.org/10.48550/arXiv.1711.06677>
- [11] H. van Hasselt, M. Hessel, and J. Aslanides, "When to use parametric models in reinforcement learning?" 2019. [Online]. Available: <https://doi.org/10.48550/arXiv.1906.05243>
- [12] Y. Pan, H. Yao, A. massoud Farahmand, and M. White, "Hill Climbing on Value Estimates for Search-control in Dyna," 2019. [Online]. Available: <https://doi.org/10.48550/arXiv.1906.07791>
- [13] R. Agarwal, M. Schwarzer, P. S. Castro, A. Courville, and M. G. Bellemare, "Deep Reinforcement Learning at the Edge of the Statistical Precipice," 2021. [Online]. Available: <https://doi.org/10.48550/arXiv.2108.13264>